

# SQUEEZE — Enterprise LLM Context Compressor

Complete Technical Architecture, Feature Matrix & Project Evolution Report

## 1. Executive Summary

**Squeeze** (formerly *TokenMaxer*) is an enterprise-grade, content-aware Chrome extension designed for maximum token efficiency, cost reduction, and low-latency interaction on AI LLM platforms including **Claude.ai**, **Gemini** ([gemini.google.com](https://gemini.google.com)), and **ChatGPT** ([chatgpt.com](https://chatgpt.com)). Squeeze prunes redundant conversational fluff, compresses JSON payloads by 60–95%, compresses code by 15–30%, manages live thread context depth, tracks subscription quotas, and extracts local PDF content without data leaks.

### Project At-a-Glance Specifications

| Attribute          | Specification & Value                                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|
| Project Name       | Squeeze (Enterprise Context Compressor)                                                                                      |
| Target Platforms   | Claude.ai, Gemini ( <a href="https://google.com">google.com</a> ), ChatGPT ( <a href="https://chatgpt.com">chatgpt.com</a> ) |
| Extension Manifest | Chrome Extension Manifest V3                                                                                                 |
| Clean Directory    | c:\AASHU DEVS\Squeeze_Clean                                                                                                  |
| Compression Ratios | JSON: 60–95%   Code: 15–30%   Prose: 20–40%                                                                                  |
| Design System      | Warm Charcoal (#191714), Terracotta (#b5603f), Amber (#ffb700)                                                               |
| Privacy Model      | 100% Local Browser Execution (Zero External Telemetry)                                                                       |

## 2. System Modules & Technical Architecture

The Squeeze engine consists of modular JavaScript modules operating seamlessly under Manifest V3:

| Module          | File Path                  | Functionality                                                                                                    |
|-----------------|----------------------------|------------------------------------------------------------------------------------------------------------------|
| JSON Crusher    | modules/json-crusher.js    | Losslessly flattens, prunes nulls/defaults, and compacts JSON structures to cut 60–95% of prompt tokens.         |
| Code Compressor | modules/code-compressor.js | Strips comments, redundant white-spaces, and verbose structures across JavaScript, Python, C++, HTML/CSS.        |
| CCR Store       | modules/ccr-store.js       | Content Context Reversibility store. Maintains reversible state maps so compressed tokens un-squeeze accurately. |

|                  |                                |                                                                                                                          |
|------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Output Shaper    | modules/output-shaper.js       | Post-processes incoming LLM outputs to strip conversational chatter and cut output token costs.                          |
| Smart Router     | SmartRouter/router-core.js     | Dynamically evaluates prompt size and content type to auto-select optimal compression modes (Squeeze, Balanced, Polish). |
| Local PDF Engine | pdf.min.js / pdf.worker.min.js | Parses local PDF documents completely client-side in Web Workers and squeezes extracted text into input.                 |

### 3. Injected UI & User Controls

Squeeze injects a unified, non-intrusive action bar into the prompt container across Claude, Gemini, and ChatGPT, directly alongside native controls without obscuring Send or Attachment buttons.

**Inline Action Buttons Breakdown:**

- **■ Undo Button:** Instantly restores the original uncompressed prompt text if optimization is reverted.
- **■ Rules & Context Vault Button:** Styled with an amber/gold border. Opens the Context Vault & Developer Rules modal.
- **■ Squeeze PDF Button:** Opens local file selector to parse, squeeze, and insert PDF document contents.
- **■ Squeeze Funnel Button:** Manually triggers instant prompt compression and fluff removal.
- **< summarise context > Button:** Summarizes the active thread context and carries it into a fresh chat.

**Real-Time Usage Bar Components:**

Positioned directly beneath the prompt input box in a glassmorphic dark container:

- **Conversation Context Tracker:** Live token count, maximum host window (200K Claude / 1M Gemini / 128K ChatGPT), percentage, and status pill (SAFE, MODERATE, HEAVY, CRITICAL).
- **Subscription Quota Tracker:** Displays 5-hour organizational quota utilization and live reset countdown on Claude, or host model specifications on Gemini/ChatGPT.
- **Dual Progress Tracks:** Visual indicator bars showing real-time capacity fill in terracotta (#b5603f) and amber (#ffb700).

### 4. Multi-Platform Support Matrix (Claude, Gemini & ChatGPT)

Squeeze automatically adapts its DOM selectors, context limits, and styling per platform:

| Platform | Host Domain       | Input Selector                            | Context Window                          |
|----------|-------------------|-------------------------------------------|-----------------------------------------|
| Claude   | claude.ai         | div[contenteditable='true'], .ProseMirror | 200,000 Tokens (Pro Quota Live API)     |
| Gemini   | gemini.google.com | rich-textarea div[contenteditable='true'] | 1,000,000 Tokens (Gemini 1.5 Pro/Flash) |
| ChatGPT  | chatgpt.com       | #prompt-textarea                          | 128,000 Tokens (GPT-4o Standard)        |

### 5. Project Evolution & Refinements Completed

**Phase 1 — Rebranding & Core Refactoring:**

Rebranded all references from 'TokenMaxer' to 'Squeeze' across manifests, scripts, popups, and stylesheets.

**Phase 2 — UI Simplification & Drawer Removal:**

Removed the invasive left sidebar button and slide-out drawer based on user feedback. Shifted all rules, context vault controls, and metrics into clean inline controls and modals.

Phase 3 — Custom Tooltip System:

Replaced native browser yellow title tooltips with sleek, dark charcoal Claude-style floating tooltips (.squeeze-tooltip).

Phase 4 — Universal Multi-Platform Adapter:

Expanded host permissions in Manifest V3 and implemented host-aware DOM query engines for Claude, Gemini, and ChatGPT.

6. Production File Directory Structure

All active, verified, production-ready code files reside in the clean standalone folder:

c:\AASHU DEVS\Squeeze\_Clean\

| File / Folder                     | Description                                                                            |
|-----------------------------------|----------------------------------------------------------------------------------------|
| manifest.json                     | Manifest V3 extension configuration & host permissions.                                |
| background.js                     | Extension background service worker for storage and messaging.                         |
| content.js                        | Primary content script with multi-platform DOM injection & compression.                |
| content.css                       | Unified design system stylesheet (terracotta, charcoal, glassmorphism).                |
| popup.html / popup.js / popup.css | Extension popup interface for mode selection & statistics.                             |
| modules/                          | Core engine scripts (json-crusher, code-compressor, ccr-store, output-shaper, router). |
| SmartRouter/                      | Smart router UI and evaluation modules.                                                |
| pdf.min.js / pdf.worker.min.js    | Local WebWorker PDF extraction libraries.                                              |